

**PROJETO INTEGRADOR DO 3º SEMESTRE
DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
– GRUPO 5**

Testes e Qualidade de Software (DEVOPS) - Entrega 1

INTEGRANTES DO PROJETO e RA'S

Danilo Oliveira de Almeida - 23011087

Katiely de Jesus Silva - 23011355

Laura Piveta Pelizzer - 23011129

Matheus Marques Quio - 23011107

São Paulo

2026

SUMÁRIO

Introdução	3
1. Qualidade de Software	3
1.1. Modelo de Qualidade de Software (Diagrama/Design)	4
1.2. Processo (Plano) de Gerenciamento de Qualidade de Software	4
1.2.1. Metas de qualidade (derivadas do item 1.1)	5
1.2.2. Papéis e responsabilidades pela qualidade	6
1.2.3. Atividades de garantia da qualidade nas 4 atividades de Sommerville	7
1.2.4. Atividades preventivas	7
1.2.5. Critérios de aceitação / Definition of Done por incremento	8
1.3. Identificação de Atributos de Qualidade da Norma 25010	8
1.4. Relatório explicando como a norma 25010 é utilizada no desenvolvimento	9
2. Teste de Software	9
2.1. Plano de Teste	10
2.2. Apresentar 4 Testes Unitários	10
2.3. Apresentar 2 Testes de Integração	11
2.4. Apresentar um Teste de Usuário (Sistema)	12
Referências Bibliográficas	13

Introdução

O Próxima Etapa é uma organização sem fins lucrativos cuja missão é democratizar o acesso à educação superior e à orientação profissional para jovens de escolas públicas. Como Projeto Integrador do 3º semestre do curso de Análise e Desenvolvimento de Sistemas da FECAP, o Grupo 5 está desenvolvendo o "App do Aluno": um aplicativo Android integrado a uma API REST e a um banco de dados, que permite ao estudante acompanhar cursos do próxima etapa, agenda/calendário, presença em palestras, certificados, testes de perfil e comunicação com a organização.

O problema que o sistema resolve é a dispersão de informações relevantes para o aluno atendido pelo Próxima Etapa, hoje distribuídas entre site institucional, redes sociais e contato direto, dificultando o acompanhamento de cursos, prazos e oportunidades. A proposta é centralizar essas informações em um aplicativo mobile simples, pensado para o público real da organização: jovens de 16 a 19 anos que, segundo a pesquisa de campo conduzida pelo grupo, raramente usam o site atual e chegam à Próxima Etapa majoritariamente pelo Instagram.

O público-alvo é o corpo de alunos da Próxima Etapa, e o objetivo do sistema é dar a esse público autonomia para acompanhar sua própria jornada educacional. As tecnologias adotadas são: aplicativo Android nativo com persistência local (SQLite/Room) no front-end; API REST em Node.js, Express e Prisma no back-end e Docker para containerização do ambiente de implantação.

O escopo desta entrega concentra-se no módulo do Aluno, preservando a estrutura de banco de dados já existente na Próxima Etapa. O módulo web administrativo previsto no edital será atendido pela camada de integração (API REST); as funcionalidades de pagamento não fazem parte do escopo. Este documento prepara o leitor para entender como qualidade e testes são aplicados ao longo do desenvolvimento: a Seção 1 aplica a norma ISO/IEC 25010 ao processo de qualidade do projeto, e a Seção 2 apresenta o plano de testes e os testes desenvolvidos.

1. Qualidade de Software

Para o App do Aluno, qualidade significa entregar as funcionalidades certas com desempenho, segurança, usabilidade, confiabilidade e manutenibilidade adequados ao contexto real da Próxima Etapa: um público jovem, conexão de internet nem sempre estável, e dados de estudantes que exigem cuidado sob a LGPD.

A qualidade do sistema é fundamentada na norma internacional ISO/IEC 25010, que define características e subcaracterísticas tanto para requisitos funcionais quanto não funcionais. O grupo já havia levantado requisitos não funcionais detalhados na Seção 5 do edital do projeto (separados por aplicativo mobile e por API/banco de dados); este documento reorganiza esses requisitos sob a lente da norma, definindo métricas e metas objetivas (Seção 1.3) e um processo com pontos de controle explícitos (Seção 1.2) para medir e garantir essa qualidade ao longo do desenvolvimento.

1.1. Modelo de Qualidade de Software (Diagrama/Design)

Característica	Exemplo no App do Aluno
Adequação Funcional	<i>Correção funcional:</i> a agenda exibe apenas os encontros dos cursos em que o aluno está inscrito, com carga horária e nº de encontros corretos
Capacidade de Interação	<i>Operabilidade e Inclusividade:</i> check-in por QR Code concluído em poucos toques (< 10 s), com contraste mínimo de 4,5:1 e rótulos de acessibilidade
Confiabilidade	<i>Tolerância a falhas:</i> presença registrada sem internet fica salva localmente (Room) e é sincronizada automaticamente ao reconectar
Segurança (Security)	<i>Confidencialidade:</i> cada aluno autenticado (JWT) acessa somente os próprios dados, com toda a comunicação via HTTPS

A Tabela apresenta o modelo hierárquico de qualidade adotado, com as 4 características principais do Modelo de Qualidade do Produto da ISO/IEC 25010:2023 e suas subcaracterísticas. Essas características foram priorizadas por concentrarem os requisitos não funcionais mais críticos e explícitos do edital: completude das funcionalidades do módulo Aluno, interface simples e acessível para um público jovem, tolerância a falhas de conexão e proteção de dados via HTTPS e conformidade com a LGPD. Para cada uma delas, a terceira coluna traz um exemplo concreto de como a característica aparece no App do Aluno.

As demais características, Eficiência de Desempenho, Compatibilidade, Manutenibilidade e Flexibilidade, seguem relevantes ao projeto e possuem metas próprias na Seção 1.2, mas não compõem a matriz de rastreabilidade dos 4 atributos da Seção 1.3. A característica Proteção (Safety), introduzida na versão de 2023, tem aplicabilidade limitada a um aplicativo educacional, que não controla riscos físicos ou ambientais; no projeto, ela se manifesta apenas na apresentação responsável dos resultados do teste de perfil, que não devem ser interpretados como diagnóstico psicológico, conforme exigido pelo edital.

1.2. Processo (Plano) de Gerenciamento de Qualidade de Software

Objetivo: garantir que cada incremento do App do Aluno atenda às metas de qualidade derivadas da ISO/IEC 25010 antes de ser entregue, por meio de pontos de controle definidos no processo de desenvolvimento.

Escopo: o plano abrange o aplicativo Android (módulo Aluno), a API REST e o banco de dados, cobrindo todos os ciclos do projeto.



<https://www.figma.com/board/KeR0g27Nng7htY4xN0HbIX/Processo-de-Qualidade---Ciclo-Completo-por-Frente?node-id=0-1&t=Usdyh7x6xeBavhpV-1>

O processo é organizado em seis fases (Planejamento, Modelagem e Design, Desenvolvimento, Revisão e Testes, Implantação e Entrega) e em raias por papel (Cliente, Planejamento, UX, Back-end, Mobile, QA, DevOps e Segurança). O código só avança para a fase de Teste após aprovação no Pull Request e só é entregue após passar pela bateria de testes; em caso de reprovação, retorna ao Desenvolvimento.

Característica	Métrica	Meta
Adequação Funcional	% de RFMs (01–15) implementados e cobertos por critério de aceitação	100% até a Entrega 2
Segurança	Nº de endpoints servidos fora de HTTPS ou com queries não parametrizadas	0
Confiabilidade	% de registros de presença sincronizados corretamente após reconexão	100%

Característica	Métrica	Meta
Capacidade de Interação	Taxa de conclusão das tarefas-chave (QR Code, inscrição em palestras/cursos, editar perfil, acessar calendário e eventos) em teste de usabilidade	≥ 90% dos participantes concluem sem ajuda
Eficiência de Desempenho	Tempo de resposta das consultas/listagens sob carga simulada	< 2s
Compatibilidade	% dos testes de integração app ↔ API passando	100%
Manutenibilidade	% de pull requests mesclados com revisão de código aprovada	100%
Flexibilidade	Ambiente (API + banco) sobe com docker compose up em máquina limpa, sem passos manuais	Em até 10 min
Proteção (Safety)	% das telas de resultado do teste de perfil com aviso de "orientação, não diagnóstico psicológico"	100%

1.2.2. Papéis e responsabilidades pela qualidade

Papel	Responsabilidade pela qualidade
Planejamento (todo o time)	Definir requisitos do MVP para apresentação do trabalho e também features extras se der tempo; e critérios de aceitação de cada incremento no GitHub Project
Mobile (Katiely)	Implementar as telas seguindo a arquitetura definida, com código comentado; corrigir os apontamentos da revisão
QA (Katiely/Laura)	Revisar Pull Requests (aprova ou reprova); escrever e executar os testes unitários, de integração e de carga; verificar se as metas de qualidade foram atingidas antes de cada entrega
Back-end / DBA (Katiely)	Implementar a API em camadas com validação de entrada e queries parametrizadas; garantir a integridade do banco (chaves, restrições e presença única por aluno + atividade + data/horário)
Cloud Native (Danilo)	Automatizar backup via script, containerizar com Docker, implantar em ambiente de demonstração e monitorar logs

Papel	Responsabilidade pela qualidade
DevOps - Teste e Qualidade de Software (Laura)	Conduzir o processo de qualidade: manter o diagrama e as metas da ISO/IEC 25010, planejar e consolidar os testes e o relatório de carga, e verificar a Definition of Done antes de cada entrega.
Segurança (Katiely)	Aplicar o checklist de LGPD (coleta mínima, sem dados reais em repositório ou demo) e revisar autenticação/HTTPS antes de cada implantação
Design e UX (Quio)	Produzir wireframes e mockups, aplicar heurísticas de usabilidade e acessibilidade, e conduzir os testes de usabilidade que medem a Capacidade de Interação.
Cliente (Próxima Etapa)	Validar e aprovar a entrega final de cada ciclo

1.2.3. Atividades de garantia da qualidade nas 4 atividades de Sommerville

Atividade (Sommerville)	O que o grupo faz
Especificação	Levantamento das features, backlog no GitHub Project e critérios de aceitação por matéria - etapa Planejamento
Projeto e Implementação	Arquitetura em camadas (app Android, API Node + Express, banco SQLite), desenvolvimento em branches por funcionalidade e revisão de código em Pull Request - etapas Desenvolvimento e Revisão
Validação	Testes unitários, de integração, de carga, de segurança e de aceitação, comparando os resultados com as metas de qualidade - etapa Teste
Evolução	Manutenção corretiva e evolutiva a partir do feedback do cliente e dos professores; o que surgir volta para o backlog do próximo ciclo - etapa Entrega

1.2.4. Atividades preventivas

- Revisão de requisitos e critérios de aceitação no Planejamento, antes de qualquer código ser escrito. Isso evita retrabalho por requisito mal entendido.

- Checklist de code review (padrão de código, nomenclatura, tratamento de erro, queries parametrizadas) obrigatório em todo Pull Request antes do merge, na etapa Revisão do processo.
- Uso exclusivo de dados fictícios ou anonimizados em desenvolvimento, testes e demonstrações. Isso previne exposição de dados pessoais, conforme a LGPD e a Seção 5.2 do edital.

1.2.5. Critérios de aceitação / Definition of Done por incremento

- Funcionalidade implementada conforme o RFM correspondente, sem erros conhecidos.
- Código revisado e aprovado em Pull Request por outro integrante do grupo.
- Teste (unitário e/ou de integração) relacionado escrito e passando.
- Requisito não funcional aplicável (Seção 5 do edital) verificado — ex.: HTTPS, tratamento de erro, contraste e rótulos de acessibilidade.
- Nenhum dado pessoal real no código, no repositório ou na demonstração.
- Documentação mínima atualizada (README ou comentário relevante no código).

1.3. Identificação de Atributos de Qualidade da Norma 25010

A tabela a seguir é a matriz de rastreabilidade dos 4 atributos priorizados na Seção 1.1, ligando um requisito/cenário real do projeto à característica e subcaracterística da norma, à métrica que o mede, à meta definida e à forma de verificação.

Requisito / Cenário	Característica (25010)	Subcaracterística	Métrica	Meta	Como verificar
Consulta e listagem de cursos/agenda	Adequação Funcional	Correção Funcional	% de RFMs 01–15 testados e aprovados	100% até E2	Testes de integração + checklist de aceitação por RFM
Registro de presença via QR Code	Capacidade de Interação	Operabilidade	Tempo médio para concluir o check-in	< 10s	Teste de usabilidade com usuários reais
Sincronização após perda de conexão	Confiabilidade	Tolerância a Falhas	% de registros sincronizados corretamente após reconexão	100%	Teste manual em modo avião + reconexão

Requisito / Cenário	Característica (25010)	Subcaracterística	Métrica	Meta	Como verificar
Autenticação e comunicação com a API	Segurança	Confidencialidade / Integridade	Nº de endpoints fora de HTTPS ou com queries não parametrizadas	0	Revisão de configuração do servidor + code review

1.4. Relatório explicando como a norma 25010 é utilizada no desenvolvimento

Na etapa de Planejamento, a norma entra antes de qualquer código ser escrito: os requisitos não funcionais da Seção 5 do edital são reorganizados como características e subcaracterísticas da ISO/IEC 25010, e cada um dos 4 atributos priorizados recebe uma métrica e uma meta objetiva (Seção 1.3), que se tornam parte do critério de aceitação do incremento.

Na etapa de Desenvolvimento, a arquitetura em camadas (controller/service/repository na API; Activities/Fragments organizados por responsabilidade no app) é uma decisão direta da característica Manutenibilidade — especificamente das subcaracterísticas Modularidade e Testabilidade —, pensada para que os testes unitários e de integração da Entrega 2 tenham componentes isoláveis para testar.

Na etapa de Revisão, o checklist de Pull Request verifica, além de padrão de código, pontos de Segurança (uso de queries parametrizadas, ausência de credenciais no código) e de Adequação Funcional (aderência ao RFM correspondente) — a norma vira, nesse momento, um checklist objetivo, não apenas um conceito teórico.

Na etapa de Teste, a norma se traduz diretamente nos tipos de teste exigidos pela Entrega 2 desta Unidade Curricular: os testes unitários e de integração verificam Adequação Funcional; o relatório de teste de carga verifica Eficiência de Desempenho; e testes específicos de reconexão e autenticação verificam, respectivamente, Confiabilidade e Segurança, e o teste de usabilidade com alunos da Próxima Etapa verifica Capacidade de Interação, em especial a operabilidade do check-in por QR Code.

Na etapa de Entrega, o teste de sistema/aceitação (Seção 2.4) verifica o produto como um todo em relação a todos os atributos priorizados simultaneamente — não apenas às funcionalidades previstas nos RFMs —, fechando o ciclo de aplicação da norma até a Apresentação Final.

2. Teste de Software

Os testes são a forma prática de verificar, com evidência, que o sistema atende às metas de qualidade definidas na Seção 1. O grupo adota testes em três níveis: unitário (componentes isolados), integração (componentes conectados, ex.: API e banco de dados) e usuário/sistema (o produto completo, do ponto de vista de quem usa). Os testes unitários e de integração, junto com o relatório de teste de

carga, são entregáveis obrigatórios da Entrega 2 (06/11/2026) desta Unidade Curricular; o teste de sistema/aceitação é apresentado na semana de 17 a 24/11/2026.

2.1. Plano de Teste

Tipo de teste	Ferramenta	Responsável	Momento (cronograma)
Testes unitários	Framework nativo do componente (Jest + Supertest contra banco de teste isolado (Docker))	QA	Semanas 1–2 (28/09–11/10)
Testes de integração	Mesmo framework, contra banco de teste isolado	QA	Semanas 3–4 (12–25/10)
Teste de carga do servidor	k6 ou Artillery contra a API	QA / DevOps	Semanas 5–6 (26/10–06/11)
Teste de sistema / aceitação	Roteiro manual com usuário real	QA + Usuário	Semana da Apresentação (17–24/11)

Critérios de sucesso, falha e correção

- Sucesso: o teste passa sem intervenção manual e o resultado obtido é igual ao resultado esperado.
- Falha: há divergência entre resultado obtido e esperado, exceção não tratada, ou timeout.
- Procedimento de correção: abrir uma tarefa referenciando o teste que falhou, corrigir na branch de desenvolvimento e reexecutar o teste antes de abrir novo Pull Request.

2.2. Apresentar 4 Testes Unitários

⚠ **A preencher** - Testes planejados - a coluna Resultado obtido será preenchida após a execução, até 06/11/2026 (Entrega 2).

Nº	Componente / função testada	Objetivo	Entrada utilizada	Resultado esperado	Resultado obtido
1	API de presença	Impedir presença duplicada no mesmo encontro	Aluno fictício ID 1, atividade ID 10, encontro 15/10/2026 19h, já registrado	Erro "presença já registrada" (HTTP 409); nenhum registro novo criado	
2	Calcular percentual de participação	Garantir o cálculo correto do dado numérico de participação	6 presenças em 8 encontros; e 0 encontros	75%; e 0% sem erro de divisão por zero	
3	Validar QR Code	Aceitar apenas QR Codes válidos e dentro do prazo	Token válido; token expirado; token com assinatura alterada	Válido retorna o ID da atividade; os outros dois são rejeitados	
4	Middleware de autenticação JWT (API)	Bloquear acesso sem autenticação válida	Requisição sem token; com token inválido; com token válido	401 nos dois primeiros casos; acesso liberado com o aluno identificado no terceiro	

2.3. Apresentar 2 Testes de Integração

⚠ **A preencher** - Testes planejados - a coluna Resultado obtido será preenchida após a execução, até 06/11/2026 (Entrega 2).

Nº	Cenário	Componentes envolvidos	Fluxo executado	Resultado esperado
1	Login e consulta de cursos do aluno	API (autenticação + cursos), Prisma, Banco SQL de teste	POST /auth/login com aluno fictício → recebe token → GET /alunos/me/cursos com o token	HTTP 200 com apenas os cursos em que esse aluno está inscrito; nenhum dado de outro aluno retornado
2	Check-in offline com sincronização posterior	App Android, API, Banco SQL	Registrar presença via QR Code com o aparelho em modo avião → reconectar → sincronização automática	Presença gravada no banco uma única vez, com status local "sincronizado" no app

2.4. Apresentar um Teste de Usuário (Sistema)

⚠ **A preencher** — esta seção será preenchida na semana da Apresentação Final (17–24/11/2026), com um aluno real ou simulado da Próxima Etapa.

Perfil do usuário: estudante da Próxima Etapa (real, com autorização, ou simulado), 16 a 19 anos, cursando o ensino médio ou cursinho pré-vestibular, usuário de smartphone Android, sem contato prévio com o app — alinhado à persona Ketlin definida na pesquisa de UX.

Cenário de uso: após participar de uma palestra, o aluno instala o app e o utiliza pela primeira vez para acompanhar seus cursos e registrar presença no próximo encontro.

Tarefas solicitadas: (1) fazer login; (2) consultar os cursos em que está inscrito; (3) verificar o próximo encontro na agenda; (4) registrar presença lendo um QR Code; (5) consultar o histórico de presença; (6) visualizar um certificado; (7) responder ao teste de perfil e ver o resultado.

Critério de aceitação: o usuário conclui ao menos 90% das tarefas sem ajuda e realiza o check-in em menos de 10 segundos (metas das Seções 1.2 e 1.3).

Resultados obtidos: [preencher após o teste]

Dificuldades encontradas: [preencher após o teste]

Melhorias identificadas: [preencher após o teste]

Referências Bibliográficas

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION. ISO/IEC 25010:2023 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. Geneva: ISO/IEC, 2023.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson Education do Brasil, 2019.

FECAP. PI_3ADS_202602_Proxima_Etapa_FINAL.pdf — Documento de briefing do Projeto Integrador "Próxima Etapa — App do Aluno", 3º Semestre de Análise e Desenvolvimento de Sistemas, 2026.